Sua parte: a API

Uma versão inicial simples para receber, validar, encaminhar e devolver dados. **Sem implementação de Qiskit ou ChromaDB.**

A responsabilidade foi delimitada pelo usuário: Pessoa 6 cuida dos contratos, endpoints, erros, fluxo, documentação e cálculo de avaliação. A equipe responsável pelos serviços cuida da IA, do banco vetorial e dos circuitos.

Arquivo principal	O que faz
main.py	Seis operações HTTP, tratamento de erros e avaliação.
contratos.py	Define os campos de entrada/saída e suas validações.
servicos.py	Ponto de conexão com a equipe; hoje contém somente um simulador.

O que permaneceu

As cinco operações exigidas no quadro de endpoints; entradas/saídas JSON; validação; OpenAPI 3.1; erros 4xx/5xx; sequência; notas 0..5 com média mínima 4; registro das avaliações e testes de sucesso/falha.

O que foi simplificado

Sem clientes Chroma/LLM, SQLite, histórico de análises, rotas duplicadas /v1, filtros, paginação, login ou configuração de múltiplos modos. As avaliações ficam em um arquivo JSONL simples, para preservar o requisito de registro.

Limite importante

A implementação atual é demonstrativa. Não existe conexão real ativa com a equipe. servicos.py será substituído quando os serviços e contratos reais forem fornecidos e você solicitar essa inclusão. Não se declara o projeto inteiro concluído.

A entrega anterior foi preservada. Esta versão é independente e não importa bancos ou avaliações antigos. Novas funcionalidades só devem entrar mediante sua solicitação.

Endpoints e contratos

Operação	Entrada	Retorno
GET /health	Nenhuma	status e mode
POST /query	input (texto)	response, sources, status e request_id
POST /search	query (texto)	chunks, citations, status e request_id
POST /documents	file, multipart	status, id, filename e mode
GET /documents	Nenhuma	documents e mode
POST /evaluate	Caso, avaliador, notas e evidências	Média, decisão, id e registro

/evaluate é o único endpoint além do quadro: atende à metodologia e ao registro de avaliações exigidos. As rotas /query e /search retornam 200. Upload e avaliação retornam 201.

Validações

input: 1..30.000 caracteres. query: 1..2.000. Campos extras são rejeitados. O código/circuito recebido é texto, nunca executado. Fontes trazem document_id, chunk_id, source e content.

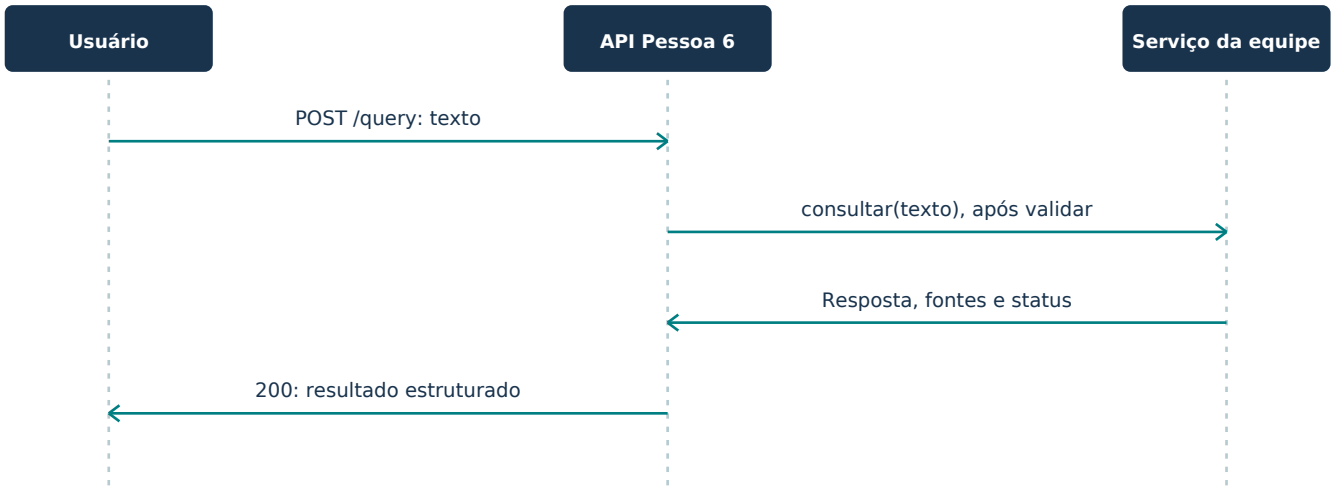
Upload: .txt, .md, .py ou .qasm, UTF-8, não vazio, até 256 KiB. A API valida a entrada e encaminha ao serviço documental. Não há extração PDF/OCR ou indexação vetorial nesta versão.

HTTP	Significado
400 / 404 / 405	Requisição malformada, rota inexistente ou método incorreto.
413 / 415 / 422	Arquivo grande, formato inadequado ou dados inválidos.
500	Falha interna ou falha ao gravar avaliação.
502 / 503 / 504	Resposta externa inválida, indisponibilidade ou timeout.

Erros retornam request_id, code, message e fields. O limite das chamadas externas é 10 segundos. Sem contexto, retorna 200 com insufficient_context, sem inventar uma conclusão técnica.

Fluxo e conexão com a equipe

A API conhece o contrato do serviço; não precisa implementar seu funcionamento interno.



No serviço externo ficam agente, RAG, ChromaDB, Qiskit e LLM. Hoje esse participante é representado por ServicosDemo. O diagrama completo, com falhas e uploads, está em docs/SEQUENCIA.md.

Método de conexão	Tarefa externa
consultar(texto)	Analisar a solicitação e gerar resposta com contexto.
buscar(query)	Recuperar documentos/trechos relevantes.
adicionar_documento(nome, bytes)	Preparar e indexar o documento.
listar_documentos()	Devolver o catálogo de documentos.

Como funciona a demonstração

O simulador começa vazio. Envie um arquivo antes de buscar. Ele guarda o texto só em memória e procura palavras de quatro ou mais caracteres, sem busca semântica. Mostra até três correspondências e até 600 caracteres de cada documento. A resposta contém SIMULAÇÃO e mode=demo.

Ao reiniciar, os documentos simulados desaparecem. Isso não é um banco vetorial. A futura integração depende de URLs, protocolo e credenciais fornecidos pela equipe; não há configuração oculta que ative uma IA real.

Metodologia de avaliação

As notas são fornecidas por um avaliador. A API valida, calcula e registra; não decide automaticamente se a IA está correta.

Critério	O que observar
Pertinência	O contexto tem relação com a entrada?
Coerência	A resposta é tecnicamente consistente?
Aderência	As fontes sustentam as afirmações?
Clareza	A explicação é organizada e compreensível?
Falhas	Limitações e incertezas são tratadas adequadamente?

Notas inteiras de 0 a 5, com evidência textual para cada critério. Proposta de âncoras: 0 não atende; 1 muito insuficiente; 2 insuficiente; 3 parcialmente adequado; 4 adequado; 5 plenamente adequado.

```
media = soma(das cinco notas) / 5
```

Condição	Decisão
Média >= 4, sem falha crítica	aprovado
Média < 4, sem falha crítica	reprovado
Falha crítica, qualquer média	revisao_obrigatoria

A revisão crítica vem do backlog. Como regra operacional proposta, nota zero em coerência, aderência ou falhas é crítica. O avaliador também pode informar critical_failure=true, com critical_evidence obrigatório.

Exemplos fictícios: [4,4,4,4,4] dá média 4 e aprova; [3,4,4,4,4] dá 3,8 e reprova; [5,0,5,5,5] dá 4, mas exige revisão. Esses números não medem a qualidade real do demo.

Registro simples

avaliacoes.jsonl é criado junto a main.py. Cada linha guarda o JSON da avaliação, com caso, avaliador, notas/evidências, média, decisão, UUID e data. O arquivo permanece após reiniciar. case_id é definido pelo avaliador, sem vínculo automático com um banco de análises.

Executar e apresentar

1. Extraia o ZIP e abra PowerShell dentro de api_pessoa6_inicial, onde estão main.py e requirements.txt. Requer Python 3.11+; validado em 3.12.

2. Execute os comandos abaixo, um por vez:

```
py -m venv .venv

.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe -m uvicorn main:app --port 8000
```

3. Mantenha o terminal aberto e acesse http://127.0.0.1:8000/docs. Em cada operação, use Try it out e Execute.

Ordem	Demonstração
1	GET /health: confirmar status ok e mode demo.
2	POST /documents: enviar exemplos/documento.txt.
3	GET /documents: mostrar o arquivo enviado.
4	POST /search com {"query":"Qiskit"}.
5	POST /query com {"input":"Qiskit"}.
6	POST /query com input vazio: demonstrar erro 422.
7	POST /evaluate: copiar exemplos/avaliacao.json.

Validação realizada

30 testes aprovados, zero falhas. OpenAPI 3.1 validado. As seis operações também foram exercitadas por HTTP real com Uvicorn. Erros de serviços e timeout foram testados com substitutos controlados. ChromaDB, Qiskit e LLM reais não foram usados.

As instruções completas estão no README. Requisitos e rastreabilidade: docs/ESPECIFICACAO.md. Evidências: docs/VALIDACAO.md e resultado_testes.txt. Especificação formal: openapi.json.